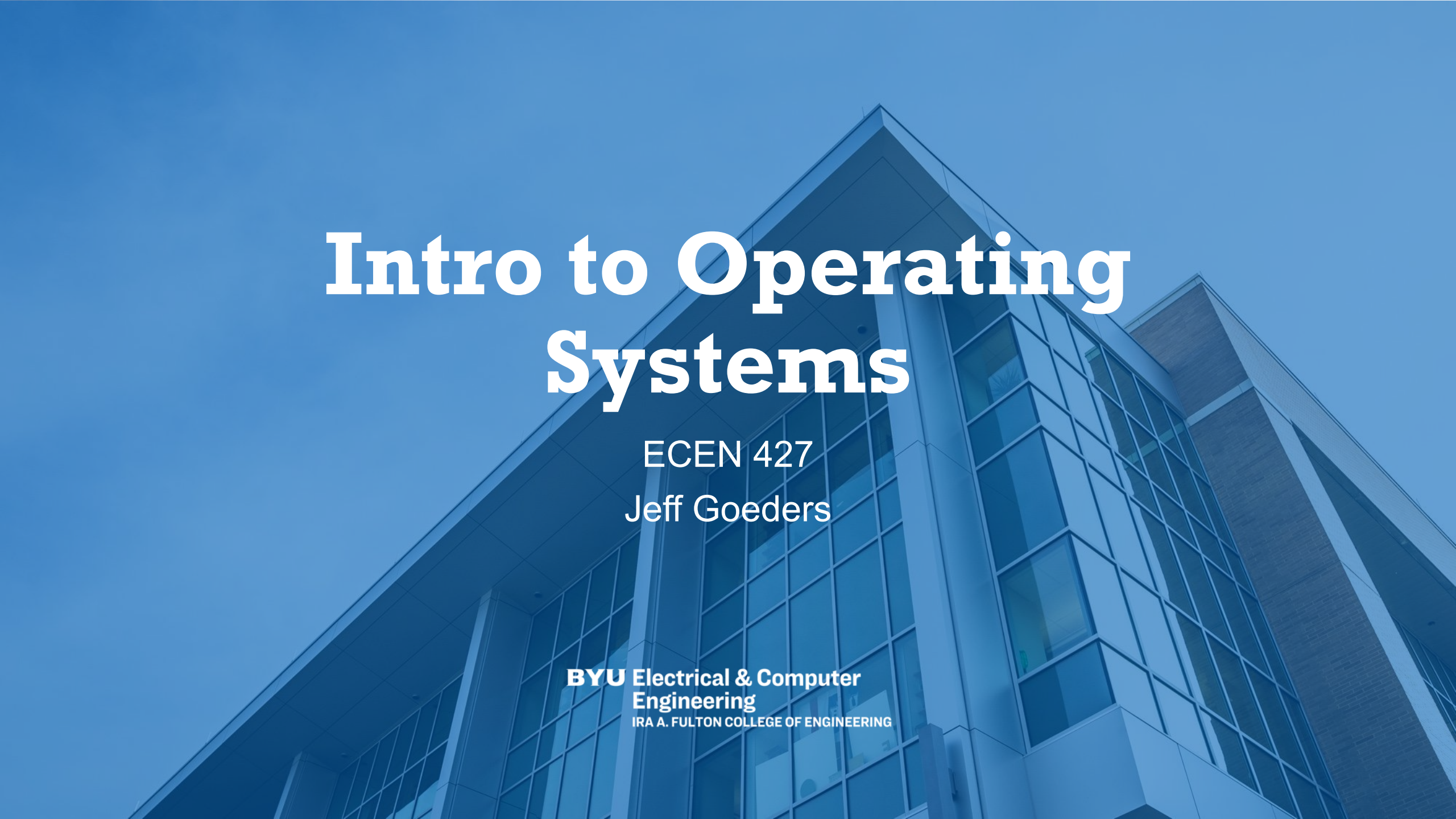


Intro to Operating Systems

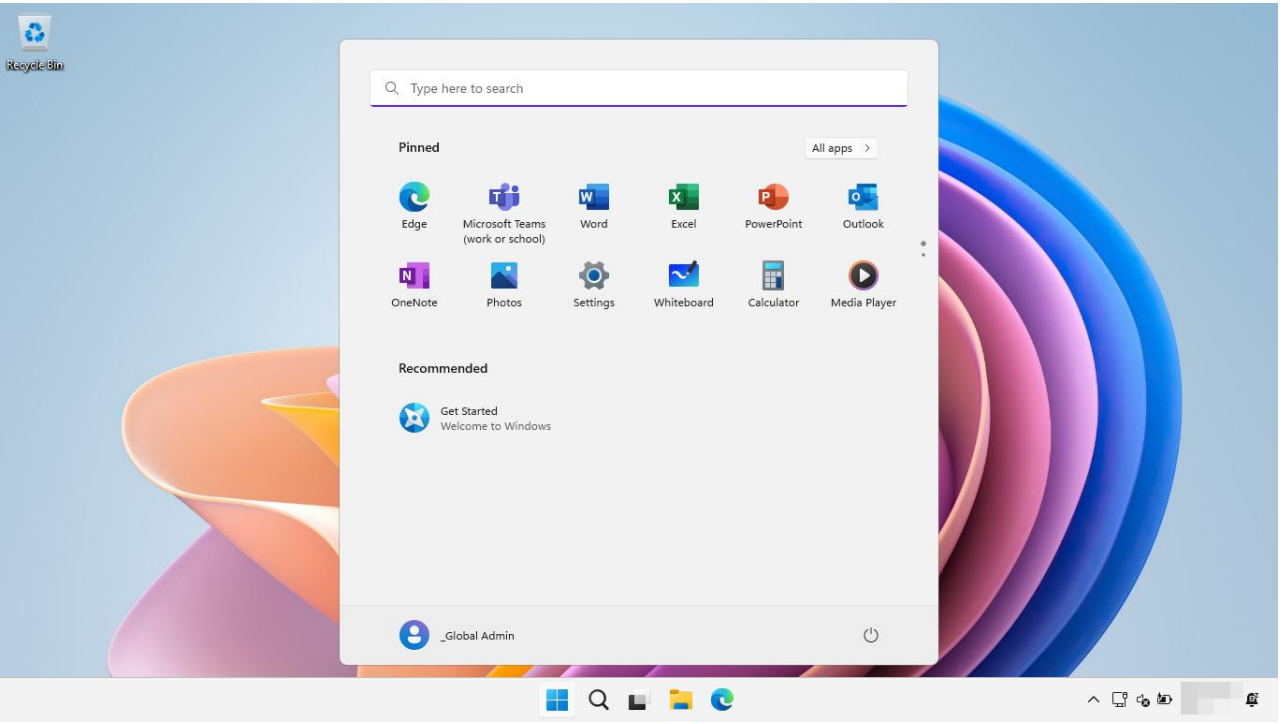
ECEN 427

Jeff Goeders

BYU Electrical & Computer Engineering
IRA A. FULTON COLLEGE OF ENGINEERING

What is an Operating System?

Windows



MacOS

Operating Systems

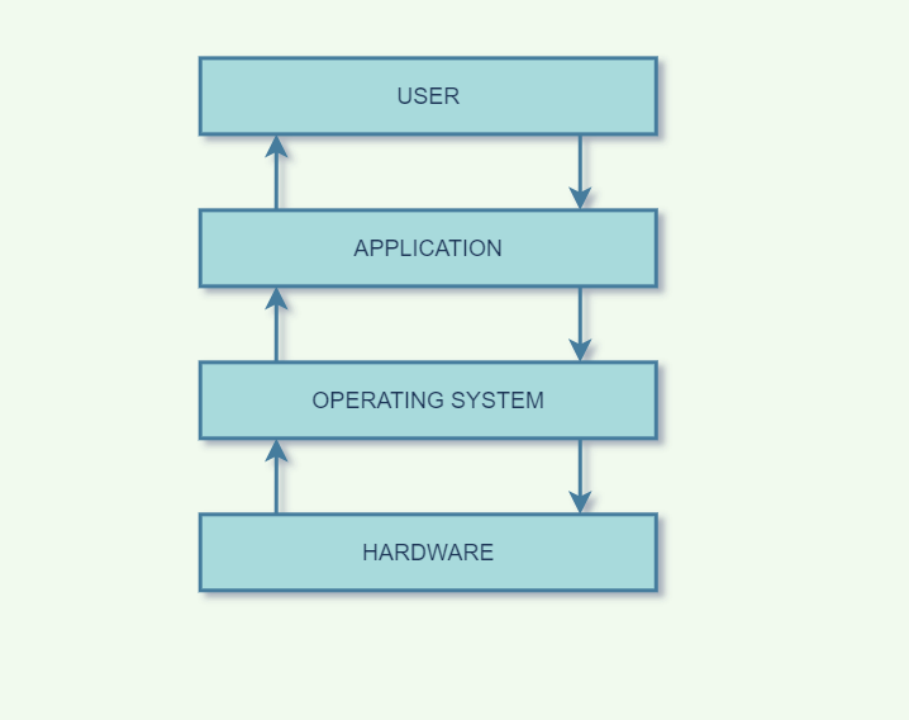
"Allows you to run multiple programs simultaneously, allowing programs to share memory, enabling programs to interact with devices, …"

"The primary way the OS does this is through a general technique that we call **virtualization**. That is, the OS takes a **physical** resource (such as the processor, or memory, or a disk) and transforms it into a more general, powerful, and easy-to-use **virtual** form of itself. Thus, we sometimes refer to the operating system as a **virtual machine**.

What does an operating system do?

1. Process management (runs programs)
2. Memory management
3. File system management
4. Device management
5. Network management

Sometimes we call this the “**kernel**”



https://www.mygreatlearning.com/blog/what-is-operating-system/

Virtualizing the CPU

Example

(instructor/lec/os/cpu.c)

./a.out & ./a.out &

killall a.out

Virtualizing Memory

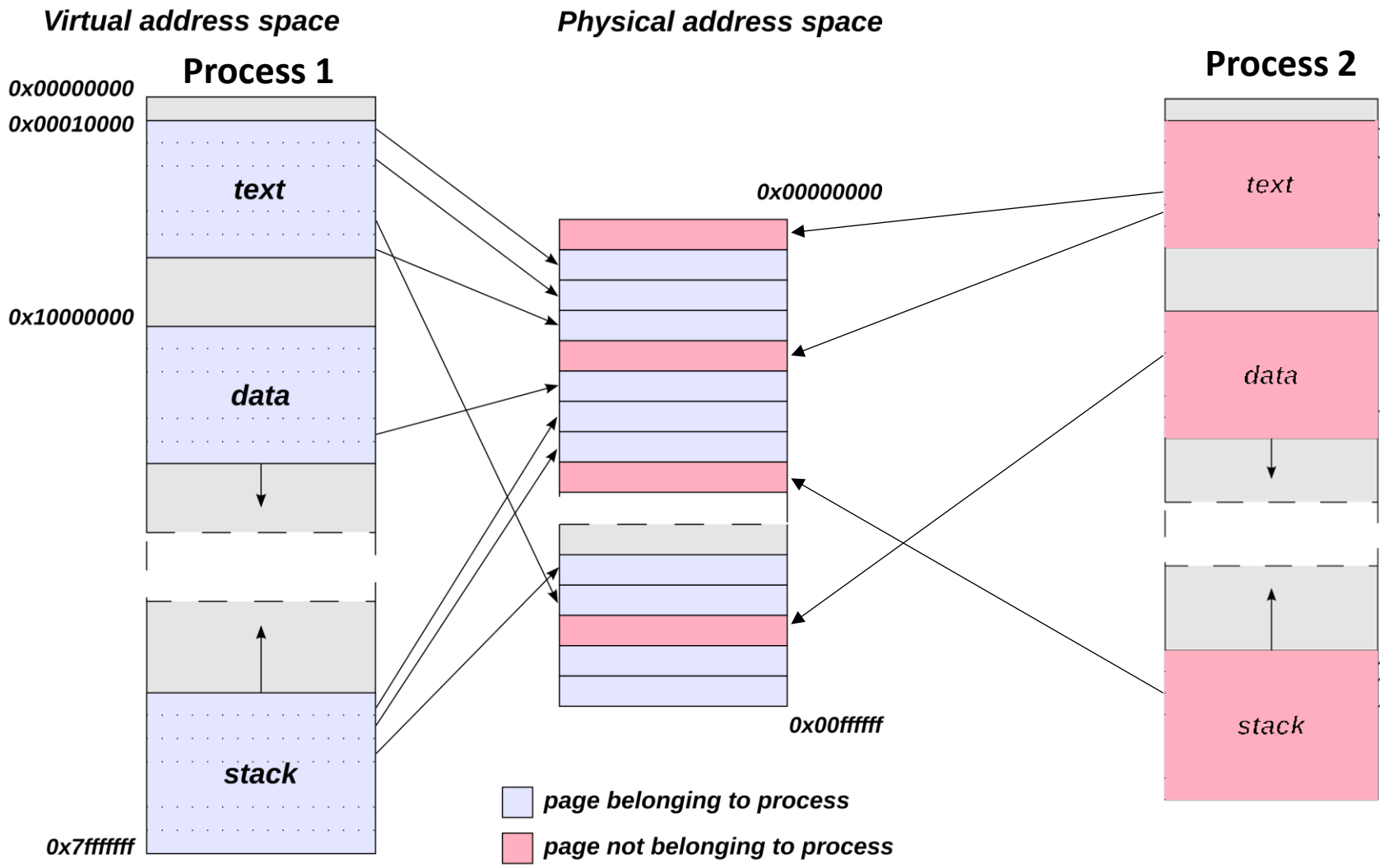
Example

(instructor/lec/os/cpu.c)

./a.out & ./a.out &

killall a.out

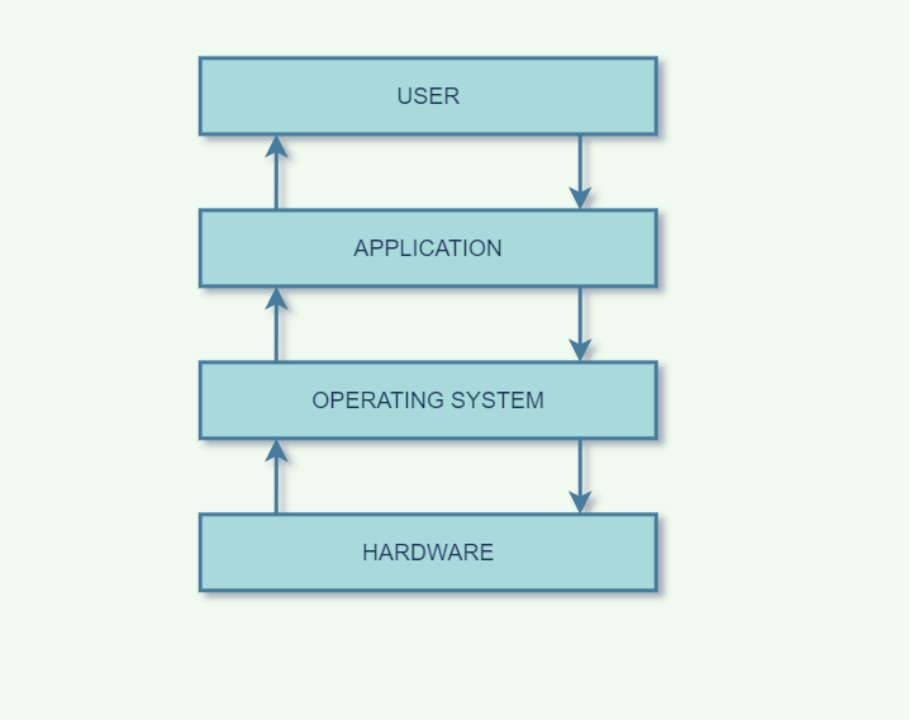
Virtualizing Memory



What does an operating system do?

1. Process management (runs programs)
2. Memory management
3. File system management
4. Device management
5. Network management

Sometimes we call this the “**kernel**”



https://www.mygreatlearning.com/blog/what-is-operating-system/

Interacting with the Operating System

Often we need to "interact" with the operating system:

1. *Process management* -> Run a program
2. *Memory management* -> Allocate memory
3. *File system management* -> Open/read/write files
4. *Device management* -> Read from a USB device
5. *Network management* -> Open a network socket

Early Operating Systems:

- Were very simplistic; just a set of available functions in a library.

- Problem: No security.
- Imagine if any program could read from any memory location, or anywhere on the disk.

Two Worlds in a Running System

- Every modern CPU has (at least) two privilege modes: user mode and kernel mode
- The hardware enforces the split, not the OS. Certain instructions and memory are simply off limits in user mode

- **Kernel space**: where the OS core runs with full hardware access
- **User space**: where every program you launch runs, inside its own sandboxed virtual address space

- Crossing between them happens only through controlled entry points (system calls, interrupts, exceptions)

Kernel vs User Space

Kernel Space

- Full privilege: can touch any memory, any device register, any CPU control state
- One shared address space for the whole kernel
- No safety net: a bug can crash the entire machine (kernel panic)
- No standard C library; different rules for memory allocation, floating point, blocking
- This is where device drivers traditionally live

User Space

- Runs with restricted privilege in a private virtual address space
- Can't touch hardware or other processes' memory directly
- Must ask the kernel for anything privileged: files, network, devices, more memory
- A crash kills only that process, the system keeps running
- Full access to libraries, debuggers, and normal development tools

User Space vs Kernel Space

	User space	Kernel space
Privilege	Restricted	Full hardware access
Memory	Private virtual address space	Shared, all of physical memory reachable
A bug crashes...	One process	The whole system
Debugging	Easy (gdb, valgrind, printf)	Hard (printk, kernel oops)
Hardware access	Only via system calls	Direct
Development	Any language, any library	C, kernel APIs only

System Calls

- A system call transfers control (i.e., jumps) into the OS while simultaneously raising the hardware privilege level
- This is done via a **trap** instruction

- When a system call is initiated, the hardware transfers program control to a **system call handler** and simultaneously raises the privilege level to kernel mode.

- When the OS is done servicing the request, it passes control back to the user via a special **return-from-trap** instruction, which reverts to user mode while simultaneously passing control back to where the application left off.

Linux System Calls

- https://www.chromium.org/chromium-os/developer-library/reference/linux-constants/syscalls/

- strace ./a.out
- strace --summary-only ./a.out

Memory Isolation

- How can we enforce that a process cannot access another processes memory?
- What if it maliciously sets a pointer to an address owned by another process and uses it?
- Key to provide protection: Virtual Memory (VM)
  - Each program has its own dedicated address space and *cannot* access memory from another program
- Hardware Memory Management Unit (MMU) added to support Virtual Memory

Virtual Memory

- Provides illusion that each program has *entire* memory to itself
  - Abstraction of private address space simplifies programs
  - Virtual memory provides a way to protect programs from each other (and from themselves!)
  - In reality, programs share physical main memory

- Key ideas
  - Program references memory using virtual addresses
  - System translates every reference to a physical address
  - Entire mechanism managed by OS with hardware assist

Virtualizing Memory